

INFORME TÉCNICO

Taller CRUD - Ejercicio 9

Fundacion Cultural "Alma Ciudad"

Aprendiz:

Andres Felipe Gonzalez Noreña

Instructor:

Richard Alexander Betancur

Fecha: Junio 2026

1. Introducción

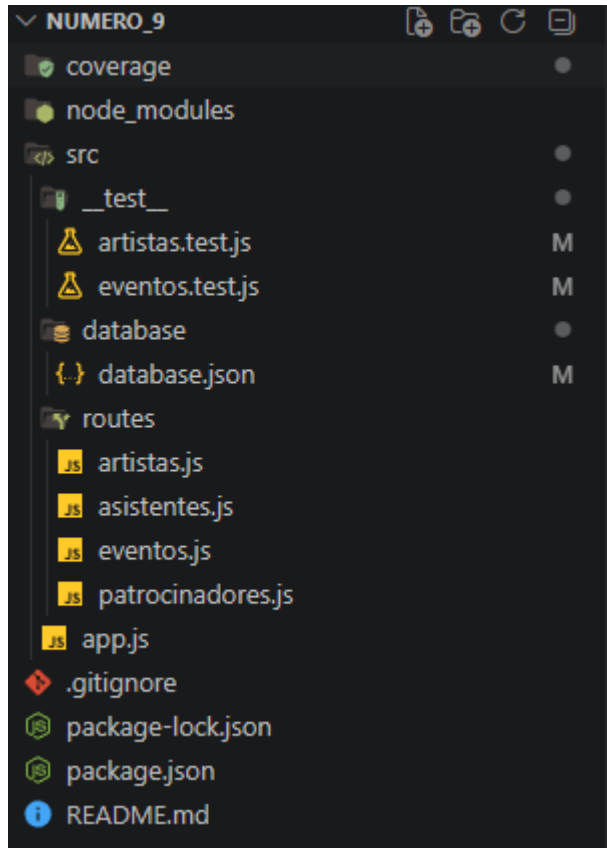
Este informe técnico documenta el diseño, la implementación y la validación de la API REST para la fundación cultural "Almas de la Ciudad". En el documento se detallan los requisitos mínimos para el funcionamiento del entorno de desarrollo, el diseño lógico de la solución mediante un diagrama de clases y las evidencias en captura de pantalla del servidor en marcha. Asimismo, se describen los resultados de las pruebas funcionales realizadas para verificar la correcta vinculación de los artistas y el control estricto de aforo de los asistentes, finalizando con un conjunto de conclusiones y recomendaciones para la futura escalabilidad del sistema.

2. Requerimientos Funcionales

- **Entorno de Ejecución:** Node.js (Versión LTS 18 o superior) y Express.js
- **Herramientas de Desarrollo:** Visual Studio Code y extensión Thunder Client.
- **Framework de Pruebas:** Jest y Supertest para la automatización.
- **Persistencia de Datos:** Archivo plano local en formato JSON (`database.json`) gestionado con el módulo nativo `fs`.
- **Estructura Base del JSON:** Un objeto inicializado con los arreglos de eventos, artistas, asistentes y patrocinadores.

3. Diseño de la Solución

El sistema fue desarrollado aplicando un patrón arquitectónico modular y desacoplado, distribuyendo las responsabilidades del backend en componentes independientes para facilitar su mantenimiento y escalabilidad.



- **src/app.js (Punto de Entrada):** Inicializa el servidor Express, configura los middlewares globales y gestiona el enrutamiento principal hacia los diferentes módulos.
- **src/routes/ (Capa de Lógica y Enrutamiento):** Contiene los controladores independientes de la API (artistas.js, asistentes.js, eventos.js, patrocinadores.js). Cada archivo encapsula las reglas de negocio y las peticiones HTTP correspondientes.
- **src/database/ (Capa de Persistencia):** Alberga el archivo plano database.json, el cual actúa como la base de datos centralizada del sistema, modificada de manera síncrona mediante el módulo fs de Node.js.
- **src/__test__/ (Capa de Pruebas):** Aloja las suites de pruebas automatizadas (artistas.test.js y eventos.test.js) que validan de forma aislada el comportamiento de los endpoints sin alterar el flujo principal de la aplicación.

4. Evidencias de Implementación

```
PS C:\Users\Andres\Documents\numero_9> node src/app.js
SERVIDOR ENCENDIDO EN: http://localhost:3000
```

TC New Request X

POST http://localhost:3000/artistas Send

Status: 201 Created Size: 191 Bytes Time: 29 ms

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "identificacion": "ART-55",
3   "nombre": "Débora Arango",
4   "disciplina": "Pintura Expresionista"
5 }
```

Response Headers 6 Cookies Results Docs

```
1 {
2   "mensaje": "Artista guardadito",
3   "artista": {
4     "identificacion": "ART-55",
5     "nombre": "Débora Arango",
6     "disciplina": "Pintura Expresionista",
7     "biografia": "Sin biografía :0",
8     "contacto": "Sin datos :("
9   }
10 }
```

Response C

TC New Request X

POST http://localhost:3000/eventos Send

Status: 201 Created Size: 315 Bytes Time: 12 ms

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "nombre": "Exposición Acuarelas de la Revolución",
3   "tipo": "exposicion",
4   "fecha": "2026-09-18T10:00:00Z",
5   "ubicacion": "Museo de Arte Moderno de Medellín (MAMM)",
6   "capacidadMaxima": 50
7 }
```

Response Headers 6 Cookies Results Docs </> ≡

```
1 {
2   "mensaje": "Evento programado con éxito",
3   "evento": {
4     "codigo": "EV-2",
5     "nombre": "Exposición Acuarelas de la Revolución",
6     "tipo": "exposicion",
7     "fecha": "2026-09-18T10:00:00Z",
8     "ubicacion": "Museo de Arte Moderno de Medellín (MAMM)",
9     "capacidadMaxima": 50,
10    "entradasVendidas": 0,
11    "estado": "programado",
12    "artistasVinculados": []
13  }
14 }
```

Response Chart ↗

TC New Request X

POST http://localhost:3000/artistas/vincular Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "codigoEvento": "EV-2",
3   "identificacionArtista": "ART-55"
4 }
```

Status: 200 OK Size: 388 Bytes Time: 8 ms

Response Headers 6 Cookies Results Docs </> ≡

```
1 {
2   "mensaje": "Artista Débora Arango vinculado con éxito al evento Exposición Acuarelas de la Revolución",
3   "evento": {
4     "codigo": "EV-2",
5     "nombre": "Exposición Acuarelas de la Revolución",
6     "tipo": "exposicion",
7     "fecha": "2026-09-18T10:00:00Z",
8     "ubicacion": "Museo de Arte Moderno de Medellín (MAMM)",
9     "capacidadMaxima": 50,
10    "entradasVendidas": 0,
11    "estado": "programado",
12    "artistasVinculados": [
13      "ART-55"
14    ]
15  }
```

Response Chart ↗

TC New Request X

POST http://localhost:3000/asistentes/registrar Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "cedula": "10203040",
3   "nombre": "Carlos Restrepo",
4   "codigoEvento": "EV-2"
5 }
```

Status: 201 Created Size: 97 Bytes Time: 9 ms

Response Headers 6 Cookies Results Docs </> ≡

```
1 {
2   "mensaje": "Asistente registrado",
3   "asistente": {
4     "nombre": "Carlos Restrepo",
5     "codigoEvento": "EV-2"
6   }
7 }
```

Response Chart ↗

5. Resultados de Pruebas Funcionales

A. Pruebas Manuales y Control de Errores (Thunder Client)

Caso de Prueba	Método	Endpoint Evaluado	Estado HTTP	Resultado Esperado en el Sistema
01. Registro Exitoso	POST	/artistas	201 Created	El artista se almacena correctamente en la lista del archivo JSON.
02. Control de Duplicados	POST	/artistas	400 Bad Request	El sistema rechaza la petición e impide registrar un artista con una identificación existente.
03. Creación de Evento	POST	/eventos	201 Created	Se genera el evento y se le asigna de forma automática su código consecutivo (EV-2).
04. Vinculación Exitosa	POST	/artistas/vincular	200 OK	El código del artista se añade al arreglo "artistasVinculados" del evento seleccionado.
05. Registro de Asistente	POST	/asistentes/registrar	201 Created	El asistente se guarda y el contador "entradasVendidas" del evento se incrementa en 1.

06. Control de Aforo	POST	/asistentes/registrar	400 Bad Request	El backend frena la transacción emitiendo un mensaje de "Capacidad máxima alcanzada".
-----------------------------	------	-----------------------	------------------------	---

B. Pruebas Automatizadas y Cobertura de Código (Jest & Supertest)

```

PS C:\Users\Andres\Documents\numero_9> npx jest --runInBand --coverage
PASS src/__test__/artistas.test.js
PASS src/__test__/eventos.test.js

-----|-----|-----|-----|-----|
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|
All files | 73.83   | 63.15    | 76.47   | 76.4    |
artistas.js | 58.13   | 64.7     | 57.14   | 57.89   | 38,43-70
eventos.js  | 84.37   | 61.9     | 90      | 90.19   | 17,37,53,74,92
-----|-----|-----|-----|-----|

Test Suites: 2 passed, 2 total
Tests:       7 passed, 7 total
Snapshots:  0 total
Time:        1.131 s, estimated 2 s
Ran all test suites.
PS C:\Users\Andres\Documents\numero_9>

```

6. Conclusiones y Recomendaciones

Conclusiones

- **Garantía en las Reglas de Negocio:** La correcta implementación de validaciones en el backend demostró que es posible controlar el aforo en tiempo real de forma automatizada. El sistema incrementa con éxito la variable de entradas vendidas y bloquea de manera segura nuevas solicitudes cuando se alcanza el límite establecido, previniendo fallos críticos de sobrecupo.
- **Modularidad y Calidad de Software:** La separación del proyecto en componentes independientes a través de una estructura limpia de rutas aisló eficazmente la lógica de cada entidad. Asimismo, el respaldo de la suite de pruebas automatizadas con Jest ejecutadas secuencialmente mitigar los riesgos de regresión, garantizando un código robusto y altamente mantenible.

Recomendaciones

- Migración del Sistema de Persistencia: Aunque el almacenamiento local en formato JSON con el módulo nativo `fs` es excelente para el entorno académico, no es escalable para producción. Se aconseja migrar hacia un motor de base de datos formal, como PostgreSQL o MongoDB, para gestionar de forma óptima la concurrencia de múltiples usuarios simultáneos.
- Integración de Middlewares de Validación: Se sugiere incorporar herramientas de validación de esquemas como Zod o Joi en una capa intermedia de Express antes de procesar las rutas. Esto permitirá limpiar la lógica del negocio, delegando en el middleware la verificación estricta de tipos de datos, obligatoriedad de campos y formatos requeridos.
- Estrategia para Incrementar Cobertura: Para elevar el porcentaje de cobertura de código por encima del 90%, se recomienda diseñar escenarios de prueba específicos dirigidos a fallos. Simular problemas controlados, como la pérdida de permisos de escritura en disco, forzará la ejecución de los bloques `catch` para validar las respuestas de error del servidor.